Análisis agradble de leer, y que denot apropaición de los conceptos del curso.

Análisis de un lenguaje de programación: Elixir

[redacted]

Universidad del Valle
Modelos y Paradigmas de Programación

Resumen

Entender como funcionan los programas escritos en un lenguaje de programación implica necesariamente entender los conceptos, los modelos, las técnicas y los paradigmas en los cuales el lenguaje de programación esta basado. Los modelos de programación declarativo, con estado, concurrente, orientado a objetos o relacional son solo algunos de los muchos que existen actualmente, pero que componen una base fundamental sobre la cual otros modelos y paradigmas son desarrollados. En este articulo se pretender abordar a Elixir desde estos modelos de programación, analizando su capacidad de implementar programas bajo cada uno de ellos.
Keywords: modelos de programación, paradigmas de programación, elixir

1. Introducción

Plataformatec, una consultora de software, creó Elixir en 2012 como un proyecto de Investigación y Desarrollo dirigido por José Valim. El objetivo de Elixir es ser un lenguaje productivo para escribir software confiable y mantenible. Elixir se ejecuta sobre la máquina virtual de Erlang, que proporciona una base escalable y tolerante a fallos. Elixir fue diseñado para aprovechar esta base sin costos de rendimiento y tiene como objetivo contribuir a un ecosistema más amplio siempre que sea posible.[5]

Como la gran mayoría de lenguajes prácticos, Elixir soporta los tipos de datos básicos, tales como enteros, flotantes, booleanos, átomos y cadenas. También soporta estructuras de datos como listas, tuplas, lista de palabras clave(listas cuyos índices son átomos) y mapas(diccionarios cuyas claves pueden ser de cualquier tipo).
La coincidencia de patrones(**pattern matching**) es una de las características mas poderosa de Elixir. Esta permite coincidir(o hacer "match") entre valores simples, estructuras de datos, e incluso funciones. Así por ejemplo, al ejecutar la siguiente instrucción:

```
iex> "Esto es " <> x = "Esto es Elixir"
```

la variable `x` tendrá el valor de **`"Elixir"`**.

Elixir se define como un lenguaje dinámico y funcional y bajo esta definición en la siguientes secciones se procederá a analizar su capacidad de soportar distintos modelos y paradigmas de programación:

En la sección 2 se analizará como Elixir puede soportar el **modelo de computación declarativo** teniendo en cuenta conceptos fundamentales de este modelo tales como *variable declarativa*, *transparencia referencial*, etc.

En la sección 3 se analizará a Elixir bajo el **modelo de computación concurrente declarativa**, la implementación del concepto de *hilo* y si la ejecución de un programa multi-hilo se realiza de forma *no-determinista no-observable*, entre otros.

En la sección 4 se analizará como Elixir soporta la implementación de programas con *estado explicito*.

En la sección 5 se analizará a Elixir bajo el **modelo de computación concurrente por paso de mensajes**.

En la sección 6 se analizará si Elixir soporta **programación orientada a objetos**, así como el concepto de *herencia*, sus limitaciones y alcances.

En la sección 7 se analizará si es posible construir programas bajo el **modelo de computación relacional** en Elixir.

En la sección 8 se procederá a sacar algunas conclusiones con respecto a todo lo anterior.

2. Programación declarativa

Elixir es un lenguaje inmutable(debe correr en la máquina virtual de Erlang, el cual, también es un lenguaje inmutable), sin embargo, las variables tienen un comportamiento particular al momento de coincidir con valores distintos. Veamos porqué la siguiente ejecución no produce un error:

```
iex> animal = "perro"
"perro"
iex> animal = "gato"
"gato"
```

La primero que habría que entender es que en Elixir el operador = es llamado el **operador de coincidencia**(match operator) y no es llamado el operador de asignación. Este es el operador básico para soportar la coincidencia de patrones que se habló en la introducción. Entonces, en Elixir la coincidencia `animal = ‘‘perro‘‘` es verdadera, y asigna el valor de `‘‘perro‘‘` a la variable `animal`. Lo que pasa después, en la segunda coincidencia `animal = ‘‘gato‘‘`, es que Elixir retorna una variable `animal` totalmente nueva que apunta al valor de `‘‘gato‘‘` y por lo tanto el valor anterior de `‘‘perro‘‘` ya no es accesible. Este proceso se conoce como **re-ligamiento**. En [6] se puede profundizar acerca de la decisión de usar el re-ligamiento, en lugar de lanzar un error y fallar el programa[2].
Lo anterior no son buenas noticias para soportar el modelo de computación declarativa, sin embargo, el lenguaje ofrece una alternativa para evitar re-ligar una variable durante una operación de coincidencia.

```
iex> animal = "perro"
"perro"
iex> ^animal = "gato"
** (MatchError) no match of right hand side value: "gato"
...
```

Después de ligar una variable a un valor, es posible usar el operador ^ para evitar que la variable sea re-ligada y se fuerce a realizar una operación de coincidencia. Este operador tiene la restricción de solo puede usarse sobre variables previamente ligadas a un valor (no puede usarse sobre una variable indefinida). Adicionalmente, las variables en Elixir deben ser inicializadas, por lo que el concepto de *valor parcial* no está plenamente soportado.

Por otra parte, Elixir soporta la programación de alto orden. Los procedimientos y funciones en Elixir son ciudadanos de primera clase y estos pueden ser embebidos en las distintas estructuras de datos como listas o mapas. Esto permite que en Elixir técnicas como la evaluación perezosa explícita puedan ser utilizadas. Elixir provee un módulo con azúcar sintáctico para la creación de flujos de datos y para definir operaciones sobre ellos de forma perezosa, por ejemplo:

```
iex> range = 1..100000
iex> stream = Stream.map(range, fn(x) -> x * 2 end)
iex> Enum.take(stream, 5)
[2, 4, 6, 8, 10]
```

Así, Elixir crea un rango de datos con los valores dados, el módulo *Stream* define operaciones perezosas sobre ellos y solo se ejecutaran a medida que sean demandados.

3. Programación declarativa concurrente

En Elixir, todo el código corre dentro de *procesos*(hilos). Los procesos son aislados unos de otros, corren concurrentemente y se comunican por paso de mensajes. La concurrencia de los procesos en Elixir es competitiva. Además, como vimos en la sección anterior, dado que las *variables declarativas* y los *valores parciales* no están plenamente soportados, en Elixir no se cuenta con el concepto de *variable de flujo de datos*, el cual es fundamental para la comunicación entre hilos en el modelo concurrente

declarativo. Sin embargo, Elixir ofrece un módulo llamado **Tasks**, el cual es construido sobre los procesos y ofrece una manera de controlarlos y supervisarlos. Así, se puede modelar programas que puedan entrar en este modelo de programación. Supongamos que se requiere hacer tres cálculos independientes entre ellos, y luego un calculo final que dependa de los tres anteriores. En Elixir se puede usar los **Tasks** con sus funciones **async** para ejecutar en procesos concurrentes, y **await** para suspender la computación, y esperar el resultado.

```
iex> a = Task.async(fn -> ... end)
iex> b = Task.async(fn -> ... end)
iex> c = Task.async(fn -> ... end)
iex> d = Task.await(a) + Task.await(b) / Task.await(c)
```

De esta manera, no importa el tiempo que tome alguno de los procesos a, b o c, la computación esperará hasta que tenga los valores para realizar el calculo d[3].

4. Programación con estado

En el modelo declarativo se mostró que las variables en Elixir pueden ser ligadas a valores distintos cada vez que se haga una operación de coincidencia cuando las variables se encuentren en la lado izquierdo de la operación. Este comportamiento permite simular el concepto de ***celda***, el cual es la capacidad de almacenar y cambiar el valor de un espacio de memoria referenciado por una variable. Sin embargo, hay que tener en cuenta que, en Elixir, al igualar dos variables, ambas variables hacen referencia al mismo valor, y si alguna de esas variables cambia de valor, la referencia de la otra no se verá alterado:

```
iex> a = [1,2,3]
[1,2,3]
iex> b = a
[1,2,3]
iex> a = 1
1
iex> b
[1,2,3]
```

5. Programación concurrente por paso de mensajes

Como vimos en la sección 3, en Elixir todo el código corre dentro de *procesos*(hilos) que se comunican por paso de mensajes. Esto parece indicar que este modelo de computación está plenamente soportado por Elixir, teniendo en cuenta que los procesos y su concurrencia son los que proveen los medios para construir aplicaciones distribuidas y tolerantes a fallos.

Los procesos en Elixir son extremadamente livianos en términos de memoria y CPU. Debido a ello, no es extraño que en una aplicación corran simultáneamente cientos de miles de procesos. Un proceso tiene un identificador (PID) único. El PID de los procesos son el ***puerto*** mediante el cual pueden recibir mensajes.

```
iex> spawn fn -> 1 + 2 end
#PID<0.43.0>
```

El código anterior crea un proceso mediante la instrucción `spawn` y retornar un PID. Los PID son otro tipo de dato, se pueden almacenar en variables, estructuras de datos, etc. Para enviar y recibir mensajes, Elixir provee las instrucciones `send` y `receive`. Cuando un proceso envía un mensaje, esta instrucción no detiene el hilo y continua ejecutando las siguientes instrucciones. Cuando un proceso recibe un mensaje, el mensaje es almacenado en la *bandeja de entrada* del proceso. Al ejecutar la instrucción `receive`, el proceso lee su bandeja entrada y, si hay mensaje, ejecuta el código definido, y si no hay mensajes, se suspende hasta que llegue un mensaje. De este manera, es posible definir procesos que iteren sobre si recursivamente, recibiendo y enviando mensajes, y adicionalmente guardando un estado interno.

```
defmodule MapaDeLlaves do
        def iniciar do
            Task.start_link(fn -> loop(%{}) end)
```

```
    end

    defp iterar(map) do
        receive do
            {:obtener, llave, proceso_que_llama} ->
                send caller, Map.get(map, llave)
                loop(map)
            {:guardar, llave, valor} ->
                loop(Map.put(map, llave, valor))
        end
    end
end
```

En el código anterior, se define un módulo el cual contiene dos funciones, `iniciar` e `iterar`. La función `iniciar`, empieza un proceso el cual llama a la función `iterar` con un mapa vacío (`%{}` es la forma de definir mapas en Elixir). La función `iterar` tiene la instrucción `receive` la cual espera hasta recibir un mensaje y ejecutar alguna de los instrucciones definidas: `{:obtener, llave, proceso_que_llama}` o `{:guardar, llave, valor}`. Dependiente del mensaje, enviar o guardar un valor en el mapa interno y realiza un llamado recursivo para volver a esperar a recibir un mensaje.[1]
Con lo anterior, podemos darnos cuenta que el concepto de *objeto puerto* está plenamente soportado.

6. Programación orientada a objetos

Elixir es un lenguaje funcional y como tal no maneja por definición el concepto de *clases*, *objetos* o *instancias de objetos*. Como vimos en la sección 4 de programación con estado, los valores en Elixir son inmutables, a pesar de que las variables pueden ser re-ligadas a otro valor distinto de su original, estas son ligadas al valor como tal, y no a una referencia en memoria. Sin embargo, dado que en Elixir se puede modelar la programación con estado (ya sea por re-ligamiento de variables o por procesos), y dada la capacidad del lenguaje para el manejo de estructuras de datos como tuplas y mapas (registros), es posible modelar la programación orientada a objetos.
En [4] es posible encontrar una implementación del modelo de programación orientado a objeto en Elixir que soporta *herencia múltiple*.

7. Programación relacional

El concepto fundamental en la programación relacional es la *escogencia no-determinista* de sentencias del programa. Elixir no implementa ni define en su núcleo este concepto, por lo que este modelo de computación no esta soportado.

8. Conclusiones

En el análisis de Elixir bajo los distintos modelos y paradigmas de programación es intuitivo darse cuenta que en este caso y posiblemente en muchos otros lenguajes, hay un modelo de programación dominante o que esta completamente soportado. Ello tiene que ver mucho con los objetivos y usos del lenguaje, para que fue hecho y cuales son los tipos de problemas que se pretende resolver con él. Elixir fue creado teniendo como una de sus premisas principales ser distribuido y tolerante a fallos y esto lo logra en el poder del modelo concurrente por paso de mensajes.
Adicionalmente, a pesar de no ofrecer instrucciones para usar el lenguaje bajo el modelo de programación orientado a objetos, nos pudimos dar cuenta que el hecho de soportar estructuras de datos como los mapas(registros), y junto con los procesos y la simulación de estado con variables que pueden ser re-ligadas, hace al lenguaje suficientemente expresivo para construir las abstracciones necesarias de modelo de programación.

Referencias

[1] Elixir. Elixir documentation. URL `https://hexdocs.pm/elixir/`.

[2] Matt Furness. Elixir variable rebinding. URL https://medium.com/everydayhero-engineering/elixir-variable-rebinding-342c5d0fd961.

[3] Kelvinst. Dataflow techniques in elixir? URL https://elixirforum.com/t/dataflow-techniques-in-elixir/8170.

[4] Wojtek Mach. Oop. URL https://github.com/wojtekmach/oop.

[5] Plataformatec. Elixir official web site. URL https://elixir-lang.org/.

[6] José Valim. Comparing elixir and erlang variables. URL http://blog.plataformatec.com.br/2016/01/comparing-elixir-and-erlang-variables/.